

Doc 10.4 — IIFE & Callback Functions

এই ডকে আমরা শিখব:

- IIFE — Immediately Invoked Function Expression
- IIFE syntax
- কেন IIFE ব্যবহার করা হতো
- Function expression + immediate execution
- IIFE with parameters
- IIFE scope/isolation
- Callback function
- Function as an argument
- Function executed later
- Synchronous callbacks
- Callback with array methods
- Callback with `setTimeout`
- Higher-Order Functions
- Callback बनाম normal function
- Callback-এর practical use
- Asynchronous JavaScript-এর foundation

1. IIFE কী?

IIFE এর পূর্ণরূপ:

Immediately Invoked Function Expression

অর্থাৎ এমন একটি function যেটা:

1. আগে define করা হয়
2. তারপর সাথে সাথেই **execute** করা হয়

Example:

```
(function () {  
    console.log("Hello");  
})();
```

Output:

```
Hello
```

এখানে function define হওয়ার পরই execute হয়েছে।

2. IIFE-এর Basic Structure

IIFE:

```
(function () {  
  // code  
})();
```

এখানে দুইটি important অংশ:

```
(function () {  
  // function  
})
```

→ Function Expression

এবং:

```
()
```

→ Immediate Invocation

3. Function Expression কেন?

সাধারণ function declaration:

```
function greet() {  
  console.log("Hello");  
}
```

এটা শুধু define করা হলো।

Execute করতে হবে:

```
greet();
```

কিন্তু IIFE:

```
(function () {  
  console.log("Hello");  
})();
```

Define + execute:

Define
↓
Immediately Execute

4. IIFE-এর আরেকটি Syntax

আরেকভাবে লেখা যায়:

```
(function () {  
  console.log("Hello");  
})();
```

দুটোই valid:

```
(function () {})();
```

এবং:

```
(function () {}());
```

Modern code-এ প্রথম syntax:

```
(function () {}());
```

বেশি দেখা যায়।

5. IIFE-এর সাথে Arrow Function

Arrow function দিয়েও IIFE করা যায়:

```
((() => {  
  console.log("Hello");  
}))();
```

Output:

```
Hello
```

6. IIFE কেন ব্যবহার করা হতো?

Modern JavaScript-এর আগে IIFE-এর সবচেয়ে বড় use case ছিল:

Private scope তৈরি করা এবং **global namespace pollution** কমানো।

পুরোনো JavaScript-এ `let`, `const`, ES Modules ইত্যাদি ছিল না।

তখন অনেক variable global scope-এ চলে যাওয়ার risk ছিল।

IIFE দিয়ে আলাদা scope তৈরি করা হতো।

7. IIFE Scope

```
(function () {  
  const secret = "private";  
  
  console.log(secret);  
})();
```

ভেতরে:

```
secret → available
```

কিন্তু বাইরে:

```
console.log(secret);
```

এটা কাজ করবে না।

কারণ `secret` IIFE-এর ভিতরের `scope`-এ আছে।

8. IIFE Scope Isolation

Mental model:

```
Global Scope
├── user
├── app
└── IIFE
    ├── secret
    ├── temp
    └── config
```

IIFE-এর ভিতরের variables বাইরে directly accessible নয়।

9. IIFE দিয়ে Temporary Variables

```
(function () {
  const firstName = "Shariar";
  const lastName = "Mahmud";

  const fullName = `${firstName} ${lastName}`;

  console.log(fullName);
})();
```

`firstName`, `lastName`, `fullName` বাইরে leak করবে না।

10. IIFE with Parameters

IIFE-তে argument পাঠানো যায়।

```
(function (name) {  
  console.log(`Hello ${name}`);  
})("Shariar");
```

Output:

Hello Shariar

Flow:

```
"IIFE call"  
  ↓  
"Shariar"  
  ↓  
name parameter  
  ↓  
function execute
```

11. Multiple Parameters

```
(function (name, age) {  
  console.log(name);  
  console.log(age);  
})("Shariar", 31);
```

Output:

Shariar
31

12. IIFE Return Value

IIFE value return করতে পারে।

```
const result = (function () {  
    return 10 + 20;  
})();
```

এখন:

```
console.log(result);
```

Output:

30

অর্থাৎ:

```
IIFE  
↓  
execute  
↓  
return 30  
↓  
result = 30
```

13. IIFE Returning Object

```
const config = (() => {  
    return {  
        apiUrl: "/api",  
        timeout: 5000  
    };  
})();
```

এখন:

```
console.log(config.apiUrl);
```

Output:

```
/api
```

14. IIFE-কে একবারের কাজ হিসেবে ভাবো

IIFE সাধারণত:

"এই code block-টা এখনই একবার চালাও।"

Example:

```
((() => {  
  console.log("Application initialized");  
})());
```

এটা function হিসেবে পরে বারবার call করার জন্য তৈরি করা হয়নি।

15. IIFE আগে বেশি গুরুত্বপূর্ণ ছিল কেন?

পুরোনো JavaScript-এ:

```
Global Scope
```

খুব সহজে variable দিয়ে ভরে যেত।

যেমন:

```
var username = "Sharir";  
var count = 10;  
var config = {};
```

অনেক script থাকলে naming conflict হতে পারত।

IIFE:

```
(function () {  
  var username = "Sharlar";  
  var count = 10;  
})();
```

এগুলোকে isolated রাখত।

16. Modern JavaScript-এ IIFE কি এখনও দরকার?

IIFE এখনও valid এবং কিছু ক্ষেত্রে useful।

কিন্তু modern JavaScript-এ alternatives আছে:

- `let`
- `const`
- Block scope
- ES Modules
- Module scope

তাই আগের মতো IIFE-এর প্রয়োজন অনেক কম।

17. IIFE vs Block Scope

আগে:

```
(function () {  
  const secret = "123";  
})();
```

Modern JavaScript-এ:

```
{  
  const secret = "123";  
}
```

দুটোতেই `secret` বাইরে accessible নয়।

তবে IIFE-এর function behavior এবং return value-এর সুবিধা আলাদা।

18. IIFE vs Function Declaration

Function declaration:

```
function greet() {  
  console.log("Hello");  
}  
  
greet();
```

IIFE:

```
(function () {  
  console.log("Hello");  
})();
```

Difference:

```
Function Declaration  
→ define  
→ wait  
→ manually call  
  
IIFE  
→ define  
→ immediately call
```

19. এখন আসি Callback Function

এটা খুব important concept।

Callback function হলো:

একটি function-কে অন্য function-এর argument হিসেবে পাঠানো, যাতে সেই function পরে বা নির্দিষ্ট সময়/পরিস্থিতিতে সেটাকে execute করতে পারে।

Example:

```
function greet(name) {  
  console.log(`Hello ${name}`);  
}  
  
function processUser(callback) {  
  callback("Shariar");  
}  
  
processUser(greet);
```

Output:

Hello Shariar

এখানে `greet` হলো callback ।

20. Function JavaScript-এ Value

JavaScript-এ function একটি value হিসেবে ব্যবহার করা যায় ।

যেমন:

```
const greet = function () {  
  console.log("Hello");  
};
```

Function-কে variable-এ রাখা যায় ।

```
const sayHello = greet;
```

Function-কে অন্য function-এ পাঠানো যায় ।

```
processUser(greet);
```

এটাই callback concept-এর foundation ।

21. Function Pass করা বনাম Function Call করা

খুব important difference:

```
processUser(greet);
```

এখানে:

```
greet
```

→ function pass করা হচ্ছে।

কিন্তু:

```
processUser(greet());
```

এখানে:

```
greet()
```

→ function immediately execute হচ্ছে।

এবং তার return value `processUser()` -এ যাচ্ছে।

22. Callback Example

```
function greet() {  
  console.log("Hello");  
}  
  
function execute(callback) {  
  callback();  
}  
  
execute(greet);
```

Flow:

```
greet
  ↓
pass as argument
  ↓
execute()
  ↓
callback()
  ↓
greet()
```

23. Callback-এর জন্য Function-এর নাম আলাদা কিছু নয়

এটা callback:

```
function greet() {
  console.log("Hello");
}

execute(greet);
```

এটাও callback:

```
execute(function () {
  console.log("Hello");
});
```

এটাও:

```
execute(() => {
  console.log("Hello");
});
```

Callback হলো **function**-এর **role**, কোনো আলাদা function type নয়।

24. Anonymous Callback

Function-এর নাম না দিয়েও callback পাঠানো যায়।

```
function execute(callback) {  
  callback();  
}  
  
execute(function () {  
  console.log("Hello");  
});
```

এখানে anonymous function callback হিসেবে পাঠানো হয়েছে।

25. Arrow Callback

Modern JavaScript-এ arrow callback খুব common:

```
execute(() => {  
  console.log("Hello");  
});
```

আর short হলে:

```
execute(() => console.log("Hello"));
```

26. Synchronous Callback

সব callback asynchronous নয়।

এটা খুব important।

Example:

```
function process(callback) {  
  console.log("Start");  
  
  callback();  
  
  console.log("End");  
}
```

```
process(() => {  
  console.log("Callback");  
});
```

Output:

```
Start  
Callback  
End
```

এটা **synchronous callback**।

Callback immediately execute হয়েছে।

27. Callback + Array Methods

JavaScript-এর অনেক array method callback নেয়।

যেমন:

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(number => number * 2);
```

এখানে:

```
number => number * 2
```

একটি callback function।

28. `map()` Callback

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.map(number => {  
  return number * 2;  
});
```

Output:

```
[2, 4, 6, 8]
```

`map()` প্রতিটি element-এর জন্য callback execute করে।

29. `filter()` Callback

```
const numbers = [1, 2, 3, 4, 5, 6];

const even = numbers.filter(number => {
  return number % 2 === 0;
});
```

Result:

```
[2, 4, 6]
```

এখানেও callback।

30. `forEach()` Callback

```
const names = ["Rahim", "Karim", "Shariar"];

names.forEach(name => {
  console.log(name);
});
```

Output:

```
Rahim
Karim
Shariar
```

31. find() Callback

```
const users = [
  { id: 1, name: "Rahim" },
  { id: 2, name: "Shariar" }
];

const user = users.find(user => user.id === 2);
```

Callback:

```
user => user.id === 2
```

32. Callback-এর Parameters

Array methods callback-এ সাধারণত multiple arguments পাওয়া যায়।

```
const numbers = [10, 20, 30];

numbers.forEach((value, index, array) => {
  console.log(value);
  console.log(index);
  console.log(array);
});
```

সাধারণত:

```
value → current element
index → current index
array → original array
```

33. Custom Callback Function

নিজের function-ও callback গ্রহণ করতে পারে।

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}
```

এখন:

```
const result = calculate(10, 5, (a, b) => {  
  return a + b;  
});
```

Result:

15

34. একই Function দিয়ে Different Behavior

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}
```

Addition:

```
calculate(10, 5, (a, b) => a + b);
```

Subtraction:

```
calculate(10, 5, (a, b) => a - b);
```

Multiplication:

```
calculate(10, 5, (a, b) => a * b);
```

একটি function-এর behavior callback দিয়ে পরিবর্তন করা যাচ্ছে।

35. Higher-Order Function

যে function:

- অন্য function-কে argument হিসেবে নেয়, অথবা
- function return করে

তাকে **Higher-Order Function (HOF)** বলা হয়।

Example:

```
function execute(callback) {  
  callback();  
}
```

এখানে `execute()` একটি higher-order function।

কারণ এটি function argument হিসেবে গ্রহণ করছে।

36. Callback vs Higher-Order Function

এগুলো গুলিয়ে ফেলো না।

```
function greet() {  
  console.log("Hello");  
}
```

`greet` হলো callback হতে পারে।

```
function execute(callback) {  
  callback();  
}
```

`execute` হলো higher-order function।

Flow:

```
execute()  
  |  
  └─ receives → greet
```

|
└─ callback

37. Function Returning Function

Higher-order function function return-ও করতে পারে।

```
function createMultiplier(multiplier) {  
  return function (number) {  
    return number * multiplier;  
  };  
}
```

এখন:

```
const double = createMultiplier(2);  
  
console.log(double(5));
```

Output:

10

এখানে `createMultiplier()` একটি function return করছে।

এটি closure concept-এর foundation।

38. Callback + `setTimeout()`

এখন asynchronous callback-এর সবচেয়ে basic example:

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Callback");  
}, 1000);
```

```
console.log("End");
```

Output:

```
Start  
End  
Callback
```

কেন?

কারণ `setTimeout()` callback-কে immediately execute করে না।

39. এখানে **Callback Later Execute** হয়

Flow:

```
console.log("Start")  
    ↓  
setTimeout()  
    ↓  
callback scheduled  
    ↓  
console.log("End")  
    ↓  
সময় শেষ  
    ↓  
callback execute
```

তাই:

```
Start  
End  
Callback
```

40. `setTimeout` **Callback** কেন **Callback**?

কারণ আমরা function-টাকে argument হিসেবে পাঠাচ্ছি:

```
setTimeout(() => {  
  console.log("Hello");  
}, 1000);
```

এখানে:

```
() => {  
  console.log("Hello");  
}
```

callback function ।

`setTimeout()` পরে এটাকে execute করে ।

41. Synchronous vs Asynchronous Callback

Synchronous

```
function execute(callback) {  
  callback();  
}  
  
execute(() => {  
  console.log("Hello");  
});
```

Callback immediately execute ।

Asynchronous

```
setTimeout(() => {  
  console.log("Hello");  
}, 1000);
```

Callback পরে execute ।

42. Important: Callback মানেই Async নয়

এটা খুব common misconception:

"Callback = asynchronous"

❌ ভুল।

Callback হতে পারে:

Synchronous

অথবা:

Asynchronous

Array methods:

```
map()  
filter()  
forEach()
```

সাধারণত synchronous callback execute করে।

`setTimeout()` callback পরে execute করে।

43. Practical Example — User Processing

```
function processUser(user, callback) {  
  const message = `Welcome ${user.name}`;  
  
  callback(message);  
}  
  
processUser(  
  { name: "Sharif" },  
  message => {  
    console.log(message);  
  })
```

```
}  
);
```

Output:

```
Welcome Shariar
```

44. Practical Example — Success/Error Callback

পুরোনো asynchronous JavaScript patterns-এ এমন style দেখা যায়:

```
function fetchData(success, error) {  
  const isSuccess = true;  
  
  if (isSuccess) {  
    success("Data loaded");  
  } else {  
    error("Something went wrong");  
  }  
}
```

Call:

```
fetchData(  
  message => console.log(message),  
  error => console.error(error)  
);
```

এই ধরনের callback-based API বোঝা Promise এবং `async/await` শেখার জন্য useful।

45. Callback Hell

অনেক nested asynchronous callback হলে code difficult হয়ে যেতে পারে।

Example:

```
getUser(user => {  
  getPosts(user, posts => {
```

```
getComments(posts, comments => {  
  getLikes(comments, likes => {  
    console.log(likes);  
  });  
});  
});  
});
```

এ ধরনের structure-কে বলা হয়:

Callback Hell

Code অনেক nested এবং difficult to maintain হয়ে যায়।

46. Callback Hell কেন Problem?

সমস্যাগুলো:

- বেশি nesting
- readability কমে
- error handling difficult
- debugging difficult
- flow বোঝা কঠিন

এ কারণেই পরে JavaScript-এ:

```
Callbacks  
  ↓  
Promises  
  ↓  
async/await
```

এসেছে/ব্যবহার জনপ্রিয় হয়েছে।

47. IIFE vs Callback

দুটো এক জিনিস নয়।

IIFE

```
(function () {  
  console.log("Run now");  
})();
```

Function:

নিজেই immediately execute

Callback

```
execute(() => {  
  console.log("Run when execute calls it");  
});
```

Function:

অন্য function-এর কাছে দেওয়া
↓
সেই function execute করবে

48. IIFE vs Callback Mental Model

IIFE

↓

"আমাকে এখনই চালাও"

Callback

↓

"আমাকে অন্য function-কে দাও,
সে যখন দরকার তখন চালাবে"

49. Callback Function vs Normal Function

একটি function নিজে callback type নয়।

একই function context অনুযায়ী callback হতে পারে।

```
function greet() {  
  console.log("Hello");  
}
```

Direct call:

```
greet();
```

এখানে `greet` সাধারণভাবে invoke হচ্ছে।

অন্য function-এ পাঠালে:

```
execute(greet);
```

এখানে `greet` callback হিসেবে কাজ করছে।

50. React-এ Callback

React code-এ callback প্রচুর ব্যবহার হয়।

Example:

```
<button onClick={() => {  
  console.log("Clicked");  
}}>  
  Click  
</button>
```

এখানে:

```
() => {  
  console.log("Clicked");  
}
```

event handler function হিসেবে React-কে দেওয়া হচ্ছে।

আর:

```
<button onClick={handleClick}>
  Click
</button>
```

এখানে `handleClick` function reference দেওয়া হয়েছে।

51. React-এর একটি Common Mistake

ভুল:

```
<button onClick={handleClick()}>
  Click
</button>
```

এখানে `handleClick()` immediately execute হয়ে যাবে।

সাধারণত চাই:

```
<button onClick={handleClick}>
  Click
</button>
```

অথবা `argument` দরকার হলে:

```
<button onClick={() => handleClick(10)}>
  Click
</button>
```

এখানে `arrow function callback` হিসেবে কাজ করছে।

52. Callback + Event Listener

Browser JavaScript-এ:

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

এখানে callback:

```
() => {  
  console.log("Clicked");  
}
```

Browser event ঘটলে callback execute করবে।

53. Callback + Array Method + React

React-এ:

```
const users = [  
  { id: 1, name: "Rahim" },  
  { id: 2, name: "Shariar" }  
];  
  
const names = users.map(user => user.name);
```

এখানে `map()` callback:

```
user => user.name
```

তারপর JSX:

```
{users.map(user => (  
  <div key={user.id}>  
    {user.name}  
  </div>  
))}
```

এখানে callback concept React development-এর core part।

54. Callback-এর মূল Pattern

```
function higherOrderFunction(callback) {  
  // কিছু কাজ  
  
  callback();  
  
  // আরও কাজ  
}
```

Call:

```
higherOrderFunction(() => {  
  console.log("Callback executed");  
});
```

এই pattern JavaScript-এর অনেক জায়গায় দেখতে পাবে।

55. Callback-এর তিনটি গুরুত্বপূর্ণ বিষয়

Callback বুঝতে এই তিনটা মনে রাখো:

1. Function একটি value

```
const greet = () => {};
```

2. Function pass করা যায়

```
execute(greet);
```

3. Function পরে execute করা যায়

```
callback();
```

অথবা asynchronous system-এর মাধ্যমে পরে।

56. Common Mistake — `callback()` vs `callback`

খুব important ।

Function pass করতে:

```
execute(callback);
```

Function immediately call করতে:

```
execute(callback());
```

Difference:

```
callback  
→ function reference  
  
callback()  
→ function invocation/result
```

57. Common Mistake — Callback-এ Return Expect

করা

```
[1, 2, 3].forEach(number => {  
  return number * 2;  
});
```

`forEach()` returned values দিয়ে নতুন array বানায় না।

যদি transformation দরকার হয়:

```
const result = [1, 2, 3].map(number => {  
  return number * 2;  
});
```

এখানে `map()` callback-এর return value ব্যবহার করে।

58. Common Mistake — Async Callback-এর Result Immediately পাওয়া

```
let result;

setTimeout(() => {
  result = "Done";
}, 1000);

console.log(result);
```

Immediately:

```
undefined
```

কারণ callback পরে execute হবে।

এই problem থেকেই asynchronous programming-এর deeper concepts আসে।

59. Practice — Output Predict

Question 1

```
(function () {
  console.log("Hello");
})();
```

Output কী?

Question 2

```
const result = (function () {
  return 10 + 20;
})();

console.log(result);
```

Output କି?

Question 3

```
function execute(callback) {  
  callback();  
}  
  
execute(() => {  
  console.log("Done");  
});
```

Output କି?

Question 4

```
function process(callback) {  
  console.log("A");  
  
  callback();  
  
  console.log("B");  
}  
  
process(() => {  
  console.log("C");  
});
```

Output କି?

Question 5

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Timeout");  
}, 0);  
  
console.log("End");
```

Output order কী?

Question 6

```
const numbers = [1, 2, 3];  
  
const result = numbers.map(number => number * 2);  
  
console.log(result);
```

Output কী?

60. Coding Challenge 1 — Custom Callback

এমন function তৈরি করো:

```
processNumber(10, callback);
```

যেখানে callback দিয়ে number-এর:

- double
- square
- triple

যেকোনো operation করা যাবে।

61. Coding Challenge 2 — Custom Filter

নিজের একটি function তৈরি করো:

```
myFilter([1, 2, 3, 4, 5], callback);
```

Callback-এর condition অনুযায়ী নতুন array return করবে।

62. Coding Challenge 3 — IIFE

একটি IIFE তৈরি করো যেটা:

1. দুটি number নেবে
2. যোগ করবে
3. result return করবে

Example:

```
const result = /* IIFE */;  
  
console.log(result);
```

Expected:

30

63. Coding Challenge 4 — Delayed Callback

একটি function তৈরি করো:

```
runLater(callback);
```

যেটা callback-কে 2 seconds পরে execute করবে।

Hint:

```
setTimeout()
```

ব্যবহার করতে পারো।

64. Interview Questions

Q1. IIFE কী?

Immediately Invoked Function Expression—যে function define হওয়ার পরই immediately execute হয়।

Q2. IIFE-এর syntax কী?

```
(function () {  
    // code  
})();
```

Q3. IIFE কেন ব্যবহার করা হতো?

মূলত private scope/isolation তৈরি এবং global namespace pollution কমানোর জন্য।

Q4. IIFE কি parameter নিতে পারে?

হ্যাঁ।

```
(function (name) {  
    console.log(name);  
})("Sharir");
```

Q5. Callback function কী?

যে function-কে অন্য function-এর argument হিসেবে পাঠানো হয় এবং receiving function সেটিকে নির্দিষ্ট সময়ে execute করে।

Q6. Callback কি সবসময় asynchronous?

না।

`map()`, `filter()`, `forEach()` synchronous callback ব্যবহার করে।

`setTimeout()` asynchronous callback-এর example।

Q7. Higher-Order Function কী?

যে function অন্য function গ্রহণ করে অথবা function return করে।

Q8. `callback` এবং `callback()` -এর **difference** কী?

```
callback  
→ function reference  
  
callback()  
→ function invocation
```

Q9. Arrow function কি **callback** হতে পারে?

হ্যাঁ।

```
numbers.map(number => number * 2);
```

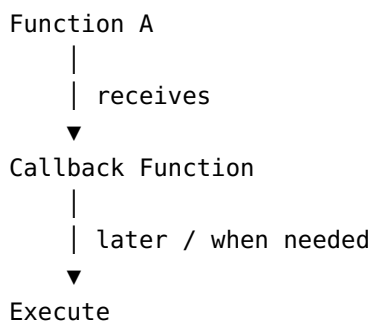
Q10. Callback Hell কী?

অনেক nested callback-এর কারণে asynchronous code যখন deeply nested এবং difficult to maintain হয়ে যায়।

65. Mental Model — IIFE

```
(function () {  
  code  
})();  
  |  
  ▼  
Function Expression  
  |  
  ▼  
Immediate Invocation  
  |  
  ▼  
Execute Now
```

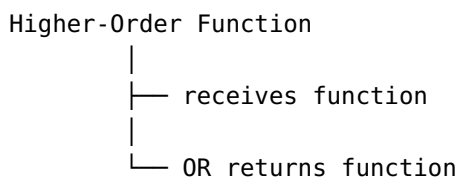
66. Mental Model — Callback



আরও সহজ:

callback
= "এই function-টা তুমি রেখে দাও,
যখন দরকার হবে তখন চালাও"

67. Mental Model — Higher-Order Function



Example:

```
function execute(callback) {  
  callback();  
}
```

68. Golden Rules ★

Rule 1

IIFE = define + immediately execute |

Rule 2

IIFE-এর main historical use ছিল scope isolation ।

Rule 3

Modern JavaScript-এ modules এবং block scope-এর কারণে IIFE-এর প্রয়োজন কমেছে ।

Rule 4

Callback হলো function-এর **role**, আলাদা কোনো function type নয় ।

Rule 5

Function JavaScript-এ value হিসেবে pass করা যায় ।

Rule 6

callback

মানে function reference ।

callback()

মানে function execute ।

Rule 7

Callback synchronous অথবা asynchronous—দুটোই হতে পারে ।

Rule 8

`map`, `filter`, `find`, `forEach` callback ব্যবহার করে ।

Rule 9

`setTimeout()` callback পরে execute করে।

Rule 10

Function গ্রহণকারী বা function returnকারী function = Higher-Order Function।

Rule 11

Callback Hell থেকে বের হওয়ার জন্য modern JavaScript-এ Promise এবং `async/await` গুরুত্বপূর্ণ।

69. Final Mastery Checklist

এই Doc শেষ করার পর তোমার পাঁচা উচিত:

- ☐ IIFE-এর full form
- ☐ IIFE syntax
- ☐ Function expression
- ☐ Immediate invocation
- ☐ IIFE parameters
- ☐ IIFE return value
- ☐ IIFE scope isolation
- ☐ IIFE-এর historical use case
- ☐ Modern JavaScript-এ IIFE-এর relevance
- ☐ Callback function কী
- ☐ Function as a value
- ☐ Function as an argument
- ☐ Callback reference বনাম invocation
- ☐ Anonymous callback
- ☐ Arrow callback
- ☐ Synchronous callback
- ☐ Asynchronous callback
- ☐ `map()` callback
- ☐ `filter()` callback
- ☐ `forEach()` callback
- ☐ `find()` callback
- ☐ `setTimeout()` callback
- ☐ Higher-Order Function
- ☐ Function returning function
- ☐ Callback Hell
- ☐ React event callback
- ☐ Callback-এর common mistakes



SECTION 10 — FUNCTIONS COMPLETE

তোমার Functions foundation এখন:

```

■ Doc 10.1
Function Fundamentals & Return Values
  ↓
■ Doc 10.2
Arrow Functions & Default Parameters
  ↓
■ Doc 10.3
Rest Parameters & Arguments Object
  ↓
■ Doc 10.4
IIFE & Callback Functions
  ↓
✅
Functions Foundation Complete
```

এরপর JavaScript-এর natural next phase হবে:

```

Functions
  ↓
Scope
  ↓
Closures
  ↓
Execution Context
  ↓
Hoisting
  ↓
this
  ↓
Call / Apply / Bind
  ↓
Higher-Order Functions
  ↓
Asynchronous JavaScript
  ↓
Promises
  ↓
async / await
```

এই flow ধরলে JavaScript-এর core concepts একে অপরের সাথে অনেক বেশি naturally connect হবে।